

1) star patterns.

square

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

logic

Row i from 1 to 5

col j from 1 to 5

Print $*$

code

```
public class square {
```

```
    psum (String[] args) {
```

```
        for (int i = 1; i <= 5; i++) {
```

```
            for (int j = 1; j <= 5; j++) {
```

```
                SOUT ("*");
```

```
            }
```

```
        SOUT ();
```

```
    }
```

```
}
```

```
}
```

2) hollow square

logic

Row i (1 to 5)

col j (1 to 5)

cond $\rightarrow$ star only print

• if ($i == 1$) or last row ($i == 5$)

• if ($j == 1$) or last col ($j == 5$)

otherwise print blank " "

```
* * * * *
*           *
*           *
*           *
* * * * *
```

public class HollowSquare

PSVM (String[] args) {

for (int i = 1; i <= 5; i++) {

for (int j = 1; j <= 5; j++) {

if (i == 1 || i == 5 || j == 1 || j == 5) {

SO UT ("X")

} else {

SO UT (" ");

}

}

SO UT (" ");

}

}

1) Triangle

```

X
X X
X X X
X X X X
X X X X X

```

logic

Row i from 1 to 5

col j from 1 to 5

• Row 1 print 1 X Row 2 print 2 X ...

• the inner loop sum upto $j \leq i$

code

Public class Triangle {

PSVM (String[] args) {

for (int i = 1; i <= 5; i++) {

for (int j = 1; j <= i; j++) {

SO UT ("X");

}

SO UT (" ");

}

}

}

public class HollowSquare

psvm (String[] args) {

for (int i = 1; i <= s; i++) {

for (int j = 1; j <= s; j++) {

if (i == 1 || i == s || j == 1 || j == s) {

 SOUT ("X");

 } else {

 SOUT (" ");

 }

 }

 SOUT (" ");

}

}

}

Triangle

X
X X
X X X
X X X X
X X X X X

logic

Row i from 1 to s

col j from 1 to s

- Row 1 print 1 X Row 2 print 1 X 2 X ... Row i print 1 X i X ...
- the inner loop runs upto $j \leq i$

code

public class Triangle {

psvm (String[] args) {

for (int i = 1; i <= s; i++) {

for (int j = 1; j <= s; j++) {

 SOUT ("X");

 }

 SOUT (" ");

}

}

}

(4) Inverted Triangle



logic

Row i from 1 to 5

col j from 1 to 5

Row 1 print 5 stars Row 2 print 4 stars

Row 5 print 1 star

formula for i : $(6-i)$

code

```
public class InvertedTri
```

```
psum (String[] args) {
```

```
for (i=1; i<=5; i++) {
```

```
for (j=1; j<=5; j++) {
```

```
    SOUT(" ");
```

```
}
```

```
SOUT("\n");
```

```
}
```

(5) pyramid

logic

spaces

Row 1 has 4 spaces

Row 2 has 3 spaces form $(5-i)$

spaces

stars

Row 1 has 1 star

Row 2 has 3 stars form $(2 \times i - 1)$

stars

code

```
public class pyramid {
```

```
psum (String[] args) {
```

```
for (int i=1; i<=5; i++) {
```

```
for (s=1; s<=(5-i); s++) {
```

```
    SOUT(" ");
```

```
}
```

```
for (int j=1; j<=(2*i-1); j++) {
```

```
    SOUT("X");
```

```
}
```

```
SOUT("\n");
```


(4) Inverted Triangle



logic

Row i from 1 to s

col j from 1 to s

Row 1 print 5 ~ Row 2 print 4 ~

~ Row 5 print 1 ~

formula for i : $(6-i)$

code

```
public class InvertedTri
```

```
psum (String[] args) {
```

```
for (i=1; i<=s; i++) {
```

```
for (j=1; j<=s; j++) {
```

```
    SOUT(" ");
```

```
}
```

```
    SOUT("\n");
```

```
}
```

(5) pyramid

logic

spaces

• Row 1 has 4 spaces

Row 2 has 3 spaces form $(s-i)$ spaces.

stars

• Row 1 has 1 star

Row 2 has 3 stars form $(2 \times i - 1)$ stars

Code

```
public class pyramid {
```

```
psum (String[] args) {
```

```
for (int i=1; i<=s; i++) {
```

```
for (s=1; s<=(s-i); s++) {
```

```
    SOUT(" ");
```

```
}
```

```
for (int j=1; j<=(2*i-1); j++) {
```

```
    SOUT("*");
```

```
}
```

```
    SOUT("\n");
```

```
}
```


(b) diamond

Total rows: 9

• split star into two halves

• upper half (same as pyramid) [1 to 5]

• lower half [5 to 1]

for privileged P running from 1 to 5,
Print (5-P) spaces & (2P-1) stars.



code

```
public class Diamond {
```

```
    sum (String args) {
```

```
        for (i=1; i<=5; i++) {
```

```
            for (j=1; j<=(5-i); j++) {
```

```
                SOUT(" ");
```

```
            }
```

```
            for (j=1; j<=(2*i-1); j++) {
```

```
                SOUT("* ");
```

```
            }
```

```
            SOUT();
```

```
        }
```

```
        for (i=4; i>=1; i--) {
```

```
            for (j=1; j<=(5-i); j++) {
```

```
                SOUT(" ");
```

```
            }
```

```
            for (j=1; j<=(2*i-1); j++) {
```

```
                SOUT("* ");
```

```
            }
```

```
            SOUT();
```

```
        }
```

(b) diamond

Total rows: 9

- split star into two halves

- upper half (same as pyramid) [1 to 5]

- lower half [4 to 1]

for previous P running from 4 to 1,
Print (s - P) spaces & (2 * P - 1) stars.



code

```
public class Diamond {
```

```
    public static void main (String[] args) {
```

```
        for (P = 1; P <= 5; P++) {
```

```
            for (J = 1; J <= (2 * P - 1); J++) {
```

```
                System.out.print(" ");
```

```
            }
```

```
            for (S = 1; S <= (2 * P - 1); S++) {
```

```
                System.out.print(" ");
```

```
            }
```

```
            System.out.println();
```

```
        }
```

```
        for (P = 4; P >= 1; P--) {
```

```
            for (J = 1; J <= (2 * P - 1); J++) {
```

```
                System.out.print(" ");
```

```
            }
```

```
            for (S = 1; S <= (2 * P - 1); S++) {
```

```
                System.out.print(" ");
```

```
            }
```

```
            System.out.println();
```

```
        }
```

```
    }
```


7) Half diamond

```

  *
 * 
* * 
* * * 
* * * * 
* * * * * 

```

logic

Rows = 9

- Upper half (1 to 5) print stars equal to the row index (i)
- Lower half (4 to 1) print stars equal to changing row index (i)

code

public class Half diamond

PSVM (String[] args) {

for (i = 1; i <= 5; i++) {

for (j = 1; j <= i; j++) {

SOOT (" ");

}

}

for (i = 4; i >= 1; i--) {

for (j = 1; j <= i; j++) {

SOOT (" ");

}

}

}

8) code

public class Half diamond Inverted.

PSVM (String[] args) {

for (i = 1; i <= 5; i++) {

for (j = 1; j <= i; j++) {

SOOT (" ");

}

for (j = 1; j <= (6 - i); j++) {

SOOT (" ");

}

SOOT ();

}

}

8) half diamond Inverted

```

* * * * *
 * * * * 
  * * * * 
   * * * 
    * * 
     * 

```

logic

Row 1 0 space, 5 stars

Row 2 1 space 4 stars

Row i (i-1) space (6-i) stars

7) half diamond

```

*
* *
* * *
* * * *
* * * * *
* * * *
* * *
* *
*
  
```

logic

Rows = 9

- Upper half (1 to 5) print stars equal to the row index (i)
- Lower half (4 to 1) print stars equal to changing row index (i)

code

```
public class half diamond
```

```
psvm (String[] args) {
```

```
for (i = 1; i <= 5; i++) {
```

```
for (j = 1; j <= i; j++) {
```

```
    SOUT(" ");
```

```
    }
```

```
    SOUT("\n");
```

```
for (i = 4; i >= 1; i--) {
```

```
for (j = 1; j <= i; j++) {
```

```
    SOUT(" ");
```

```
    }
```

```
    SOUT("\n");
```

```
}
```

```
public class half diamond Inverted.
```

```
psvm (String[] args) {
```

```
for (i = 1; i <= 5; i++) {
```

```
for (j = 1; j <= i; j++) {
```

```
    SOUT(" ");
```

```
    }
```

```
for (j = 1; j <= (5 - i); j++) {
```

```
    SOUT(" ");
```

```
    }
```

```
    SOUT("\n");
```

```
}
```

```
}
```

8) half diamond Inverted

```

* * * * *
* * * *
* * *
* *
*
  
```

logic

Row 1 0 space, 5 stars

Row 2 1 space 4 stars

Row i (i-1) space (5-i) stars

8) code

① square pattern

logic

```
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
```

Row 5

col 5

nested loop &

print 1

```
public class square {
    public static void main(String[] args) {
        for (i=1; i<=5; i++) {
            for (j=1; j<=5; j++) {
                System.out.print("1 ");
            }
            System.out.println();
        }
    }
}
```

② incrementing.

logic

code

```
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
```

• printed value

increment from 1 to 5

• printing value depends on
on inner loop

code

```
public class Increment {
    public static void main(String[] args) {
        for (i=1; i<=5; i++) {
            for (j=1; j<=5; j++) {
                System.out.print(j + " ");
            }
            System.out.println();
        }
    }
}
```


2) Number patterns.

① square pattern

logic

```

1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
    
```

Rows
cols
nested loops
print 1

code

```

public class square {
    public static void main(String[] args) {
        for (i=1; i<=5; i++) {
            for (j=1; j<=5; j++) {
                System.out.print("1 ");
            }
            System.out.println();
        }
    }
}
    
```

② Incrementing.

logic

code

```

1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
    
```

- printed value increment from 1 to 5
- printing value depends on inner loop

code

```

public class Increment {
    public static void main(String[] args) {
        for (i=1; i<=5; i++) {
            for (j=1; j<=5; j++) {
                System.out.print(j + " ");
            }
            System.out.println();
        }
    }
}
    
```


(3) Right Triangle

1
1 2
1 2 3
1 2 3 4
1 2 3 4 5

logic

Row 1 print 1 col
" 2 " 2 col
Row P " P col
(j <= P)

code

```
public class RightTriangle {  
    psuM (String[] args) {  
        for (i = 1; i <= 5; i++) {  
            for (j = 1; j <= i; j++) {  
                sOUT(j + " ");  
            }  
            sOUT();  
        }  
    }  
}
```

(4) Inverted diamond

3 3 3 3
4 4 4 4
5 5 5
6 6
7

logic

Row = 5
o Row 1 (P=3) print five 3's
o Row 2 (P=4) print 4 4's
o Row P print (8-P) e/o.

code

```
public class InvertedDiamond {  
    psuM (String[] args) {  
        for (P = 3; P <= 7; P++) {  
            for (j = 1; j <= (8-P); j++) {  
                sOUT(P + " ");  
            }  
            sOUT();  
        }  
    }  
}
```


(3) Right Triangle

logic

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Row 1 print 1 col
" 2 " 2 col
Row i " i col
($j \leq i$)

code

```
public class RightTriangle {
```

```
    psuM (String[] args) {
```

```
        for (i = 1; i <= s; i++) {
```

```
            for (j = 1; j <= i; j++) {
```

```
                sout(j + " ");
```

```
            }
```

```
            sout();
```

```
        }
```

(4) Inverted diamond

logic

```
3 3 3 3
4 4 4 4
5 5 5
6 6
7
```

Row = 5

- Row 1 ($i=3$) print five 3's
- Row 2 ($i=4$) print 4 4's
- Row i prints $(8-i)$ e/o.

code

```
public class InvertedDiamond {
```

```
    psuM (String[] args) {
```

```
        for (i = 3; i <= 7; i++) {
```

```
            for (j = 1; j <= (8-i); j++) {
```

```
                sout(i + " ");
```

```
            }
```

```
            sout();
```

```
        }
```


15) sandwich

3 3 3 3 3
4 4 4
5
4 4 4
3 3 3 3 3

logic

- Row 1 & Row 5 print five 3's
- Row 2 & Row 4 print four 4's
- Row 3 print one 5 (with two lead spaces)

6 split the logic by
calc lead spaces first
then print num.

- k run from -2 to 2
abs val |k| determines space
6 num print s - |k|

code

```
public class sandwich {
```

```
    print(String[] args) {
```

```
        for (int k = -2; k <= 2; k++) {
```

```
            int spaces = 2 - Math.abs(k);
```

```
            int count = (Math.abs(k) - 2) + 1;
```

```
            int num = 5 - Math.abs(k);
```

```
            for (int s = 0; s < spaces; s++) {
```

```
                System.out.print(" ");
```

```
            }
```

```
            for (int j = 0; j < count; j++) {
```

```
                System.out.print(num);
```

```
            }
```

```
            System.out.println();
```

```
        }
```

```
    }
```

```
}
```

6) new

3
4 4
5 5 5
6 6 6
7 7 7
8 8
9 9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

10

P.

5) sandwich

logic

3 3 3 3 3
 4 4 4
 s
 4 4 4
3 3 3 3 3

- Row 1 & Row 5 print five 3's
- Row 2 & Row 4 print three 4's
- Row 3 print one s (with two lead spaces)
- Split the logic by
calc lead spaces first
then print num.
- k run from -2 to 2
abs val |k| determines space
num print s - |k|

code

public class sandwich

psum(string[] args)

for (int k = -2; k <= 2; k++)

 int spaces = 2 - Math.abs(k);

 int count = (Math.abs(k) - 2) + 1;

 int num = s - Math.abs(k);

 for (int s = 0; s < spaces; s++) {
 System.out.print(" ");

 }

 for (int j = 0; j < count; j++) {
 System.out.print(num);

 }

 System.out.println();

}

}

}

6) Increment diamond

3
4 4
5 5 5
6 6 6 6
7 7 7 7 7
8 6 6 6
5 5 5
4 4
3

logic

row = 9

• Upper half (1 to 5)

Row value go from 3 to 7

the no of ele match program (P-2)

• lower half (6 to 9)

Row value go from 6 to 3

no ele match (P-2)

handle by same loop count.

code

public class Increment diamond

psvm (main[] args) {

for (i = 3; i <= 7; i++) {

int ele = i - 2;

for (j = 1; j <= ele; j++) {

cout (i + " ");

}

}

for (i = 6; i >= 3; i--) {

int ele = i - 2;

for (j = 1; j <= ele; j++) {

cout (i + " ");

}

cout ();

}

6) Increment diamond

logic

Row = 9

3
4 4
5 5 5
6 6 6 6
7 7 7 7 7
8 6 6 6
5 5 5
4 4
3

• Upper half (1 to 5)

Row values go from 3 to 7

the no of ele matches program (P-2)

• Lower half (6 to 9)

Row val go from 6 to 3

no ele match (P-2)

handle by same loop count.

code

```
public class IncrementDiamond {
```

```
    public static void main (String[] args) {
```

```
        for (i = 3; i <= 7; i++) {
```

```
            int ele = i - 2;
```

```
            for (j = 1; j <= ele; j++) {
```

```
                System.out.print(" ");
```

```
            }
            System.out.println();
```

```
        }
```

```
        for (i = 6; i >= 3; i--) {
```

```
            int ele = i - 2;
```

```
            for (j = 1; j <= ele; j++) {
```

```
                System.out.print(" ");
```

```
            }
```

```
            System.out.println();
```

```
        }
```

```
    }
```

```
}
```